

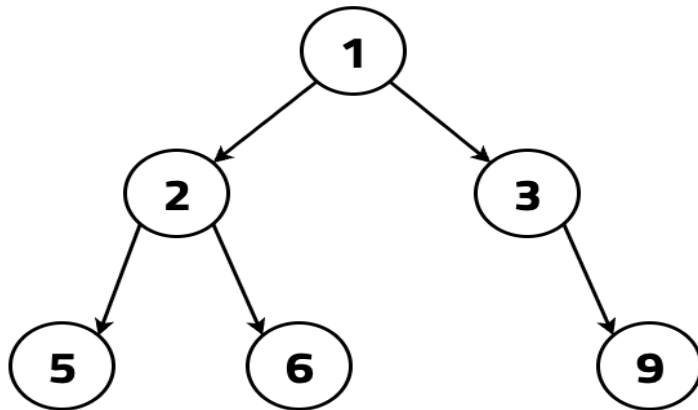
Maximum Width of a Binary Tree

Problem Statement:

Given a Binary Tree, return its maximum width. The maximum width of a Binary Tree is the maximum diameter among all its levels. The width or diameter of a level is the number of nodes between the leftmost and rightmost nodes.

Examples

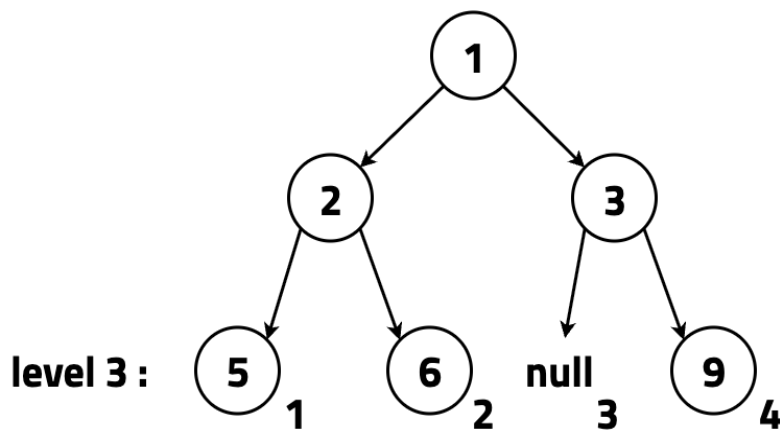
Input: Binary Tree: 1 2 3 5 6 -1 9



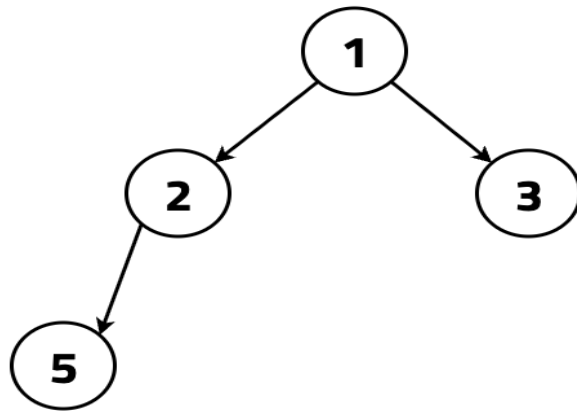
Output: Maximum Width: 4

Explanation:

Level 3 is the widest level of the Binary Tree and whose end-to-end width is 4 comprising of nodes: {5, 6, null, 9}.



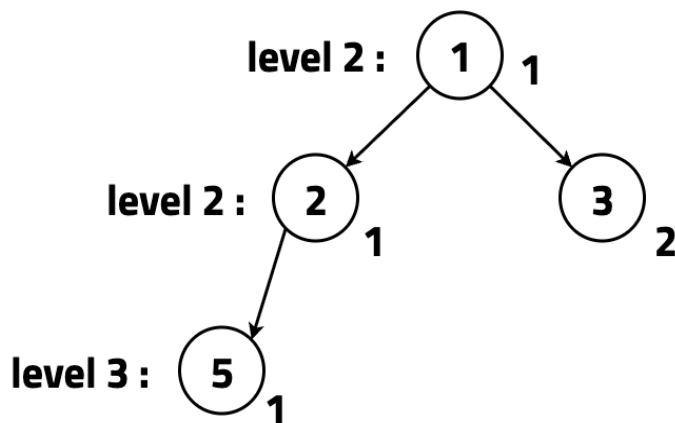
Input: Binary Tree: 1 2 3 5



Output: Maximum Width: 2

Explanation:

Level 2 is the widest level of the Binary Tree and whose end-to-end width is comprised of nodes: {2, 3}.



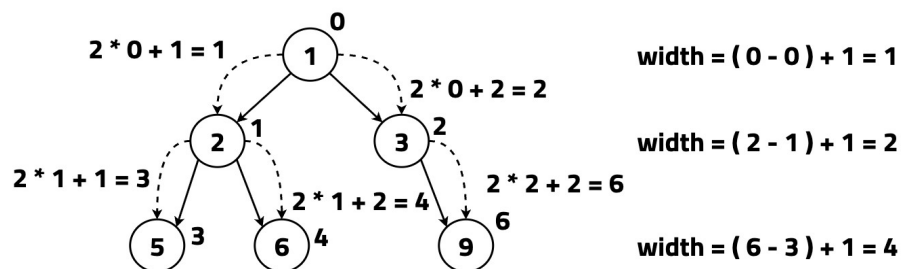
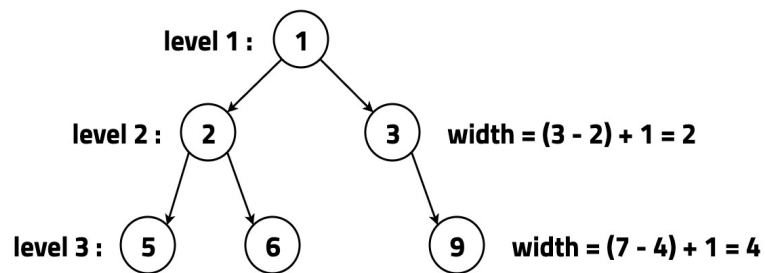
Approach

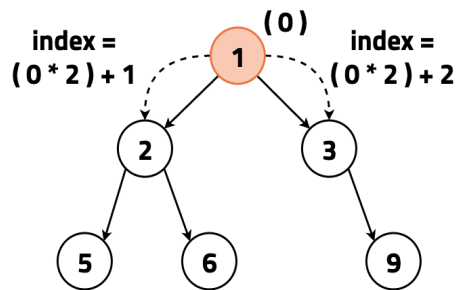
Algorithm

To determine the maximum width of a tree, an effective strategy would be to assign and identify indexes for the leftmost and rightmost nodes at each level. Using these indexes, we can calculate the width for each level by subtracting the index of the leftmost node from that of the rightmost node. Start by assigning an index to the root node as 0. For each level, the left child gets an index equal to $2 * \text{parent index}$, and the right child gets an index equal to $2 * \text{parent index} + 1$. Using a level order traversal, we use the leftmost and rightmost nodes at each level to get the width at that level. Keep track of the maximum width encountered during the traversal. Whenever a wider level is found, update the maximum width.

- Use level-order traversal (BFS) and track each node's position as if the tree were a complete binary tree.

- For each level in the tree, note the position of the first and last node and compute the width as the difference between these two positions.
- For the next level, assign positions to children. After processing all levels, return the maximum width recorded.





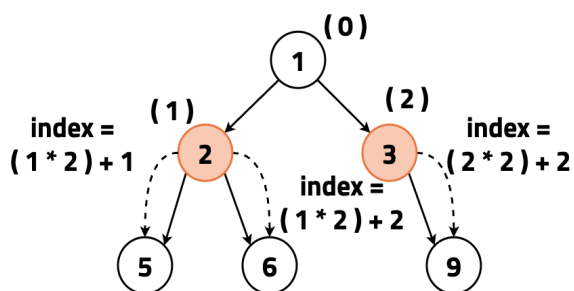
Enqueue the root node into the queue with its index as 0

[1, 0]

queue : (Node, index)

pop nodes from current level : [1, 0]

width = $(0 - 0) + 1 = 1$
ans = 1



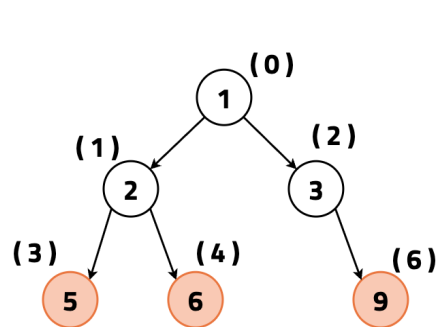
Enqueue their children into the queue with their index

[2, 1] [3, 2]

queue : (Node, index)

pop nodes from current level : [2, 1] [3, 2]
first last

width = $(2 - 1) + 1 = 2$
ans = 2



Enqueue their children into the queue with their index

[5, 3] [6, 4] [9, 6]

queue : (Node, index)

pop nodes from current level : [5, 3] [6, 4] [9, 6]
first last

width = (6 - 3) + 1 = 4
ans = 4

Code

```
import java.util.*;

// Class definition for binary tree node
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    // Constructor
    TreeNode(int val) {
        this.val = val;
        this.left = null;
        this.right = null;
    }
}

// Class containing the method
class Solution {
    // Function to find the maximum width of binary tree
    public int widthOfBinaryTree(TreeNode root) {

        // If the root is null, return 0
        if (root == null)
            return 0;

        // Initialize max width
        int maxWidth = 0;

        // Queue to store node and its index
        Queue<Pair<TreeNode, Integer>> q = new LinkedList<>();

        // Push root with index 0
        q.offer(new Pair<>(root, 0));

        // BFS traversal
        while (!q.isEmpty()) {

            // Get size of current level
            int size = q.size();

            // Get the minimum index at this level
            int minIndex = q.peek().getValue();

            // Variables to store first and last index
            int first = 0, last = 0;

            // Traverse all nodes in this level
            for (int i = 0; i < size; i++) {

                // Get current node and index
                Pair<TreeNode, Integer> p = q.poll();
                TreeNode node = p.getKey();
                int currIndex = p.getValue() - minIndex;

                // Update first index
                if (i == 0)
                    first = currIndex;

                // Update last index
                if (i == size - 1)
                    last = currIndex;
            }
        }

        return maxWidth;
    }
}
```

```

        // Push left child if exists
        if (node.left != null)
            q.offer(new Pair<>(node.left,
                               2 * currIndex + 1));

        // Push right child if exists
        if (node.right != null)
            q.offer(new Pair<>(node.right,
                               2 * currIndex + 2));
    }

    // Update maximum width
    maxWidth = Math.max(maxWidth, last - first + 1);
}

// Return the final result
return maxWidth;
}
}

// Driver class
public class Main {
    public static void main(String[] args) {

        // Create the tree
        TreeNode root = new TreeNode(1);
        root.left = new TreeNode(3);
        root.right = new TreeNode(2);
        root.left.left = new TreeNode(5);
        root.left.right = new TreeNode(3);
        root.right.right = new TreeNode(9);

        // Create solution object
        Solution sol = new Solution();

        // Call the function and print result
        System.out.println("Maximum width: " +
                           sol.widthOfBinaryTree(root));
    }
}

```

Complexity Analysis **Time Complexity: $O(N)$** , each node is processed once in level order traversal.
Space Complexity: $O(N)$, in worst case, the queue holds all the nodes of the last level i.e. $N/2$ nodes.